Manual técnico

Descripción y objetivo

La aplicación permite cargar archivos CSV de pruebas específicas de aspirantes a la Facultad de Ingeniería de la Universidad de San Carlos de Guatemala, validar sus registros y consultar resultados mediante estadísticas y relaciones. MongoDB almacena las evaluaciones y ejecuta las agregaciones; Neo4j representa sus conexiones con aspirantes, materias, carreras y procedencia educativa.

Cada fila de origen representa una evaluación. Un aspirante puede tener varias materias, oportunidades y ciclos de ingreso. Una evaluación aprobada no acredita el ingreso oficial a la universidad.

Tecnologías y arquitectura

Componente	Implementación
Backend	Java 21, Spring Boot 4.1.2, Maven Wrapper, Spring Web MVC, Spring Data MongoDB, Spring Data Neo4j y Jakarta Validation
Lectura de CSV	Apache Commons CSV 1.14.1
Frontend	Angular 21, TypeScript 5.9, componentes standalone, Reactive Forms, HttpClient, signals y RxJS
Interfaz y visualización	PrimeNG 21, CSS, gráficas HTML/CSS y SVG; Cytoscape.js 3.34.4 para grafos
Persistencia	MongoDB 7.0.45 y Neo4j 5.26 Community
Servicios locales	Docker Compose
Pruebas	Pruebas Java con los starters de Spring Boot, Vitest y verificaciones independientes en Python

Angular nunca se conecta directamente a las bases de datos. Los controladores reciben y validan solicitudes; los servicios coordinan las operaciones; los repositorios ejecutan agregaciones MongoDB o consultas Cypher. La API expone DTO, es decir, respuestas destinadas al intercambio de datos con la interfaz.

Requisitos y estructura

Se necesitan Docker Engine con Docker Compose, JDK 21, Node.js 24, npm y Bash para los comandos de configuración. Maven está incluido mediante bd2-api/mvnw. Python 3 es opcional para los scripts de comprobación; no es necesario para ejecutar la aplicación.

Ruta desde la raíz	Contenido relevante
bd2-api/	Backend, pruebas y Maven Wrapper
bd2-api/src/main/java/com/mvtech/bd2/features/	Funcionalidades dataimport, analytics, graph, search y status

bd2-api/src/main/resources/	application.yaml, estructuras de bases de datos en db/ y consultas en graph/
bd2-client/src/app/	Pantallas en features/, navegación en layout/, servicios en core/ y componentes comunes en shared/
bd2-client/proxy.conf.json	Proxy de desarrollo hacia la API
datas/ArchivosCSV/	Diez CSV locales correspondientes a ingreso 2015–2024
datas/analisis/	Scripts de perfil y contraste de resultados
docs/consultas/	Catálogos y scripts MongoDB independientes
compose.yaml, .env.example	Servicios de bases de datos y plantilla de configuración

Configuración e instalación local

Todos los comandos parten de la raíz del proyecto, salvo indicación expresa. Si ya existe un .env configurado, conservarlo; la copia siguiente corresponde a una instalación nueva:

```
cp .env.example .env
chmod 600 .env
```

Completar las contraseñas antes de iniciar los motores. La plantilla recomienda valores hexadecimales, compatibles con la URI de MongoDB y con source de Bash. No publicar .env ni incluirlo en Git.

Variable	**Uso**
MONGO_USERNAME	Usuario administrador inicial de MongoDB; la plantilla propone bd2_admin
MONGO_PASSWORD	Contraseña local de MongoDB, sin valor en la plantilla
MONGO_DATABASE	Base usada por la API; la plantilla propone bd2_final
NEO4J_USERNAME	Usuario de Neo4j; conservar neo4j
NEO4J_PASSWORD	Contraseña local de Neo4j, sin valor en la plantilla; mínimo ocho caracteres
BD2_IMPORT_DIRECTORY	Directorio de copias CSV, resultados y rechazos; por defecto .local/imports relativo al proceso

application.yaml configura MongoDB en 127.0.0.1:27017, con autenticación en admin, y Neo4j en bolt://127.0.0.1:7687. Las credenciales proceden del entorno. Los puertos no son variables propias de la plantilla.

Bases de datos

```
docker compose up -d --wait --wait-timeout 180 mongodb neo4j
docker compose ps
```

El proyecto Compose se llama bd2-final; sus servicios son mongodb y neo4j. Se utilizan las imágenes mongo:7.0.45-jammy y neo4j:5.26-community. MongoDB publica el puerto 27017; Neo4j publica 7474 para Browser y 7687 para Bolt, todos ligados a 127.0.0.1.

Los volúmenes declarados son mongodb_data, mongodb_config y neo4j_data. Las credenciales iniciales se aplican al crear los volúmenes: editar .env posteriormente no modifica las contraseñas almacenadas.

Backend

En una terminal desde la raíz:

```
java -version
set -a
source ./.env
set +a
export BD2_IMPORT_DIRECTORY="$PWD/.local/imports"
cd bd2-api
./mvnw spring-boot:run
```

La API queda disponible en http://localhost:8080. Spring Boot no carga automáticamente .env; por eso se exportan sus variables. La ruta absoluta de importaciones evita que el historial cambie según el directorio de ejecución. Una sola instancia del backend puede utilizar ese almacén a la vez.

Frontend

En otra terminal desde la raíz:

```
cd bd2-client
npm ci
npm start
```

Abrir http://localhost:4200. Angular redirige /api/** a http://127.0.0.1:8080 mediante su proxy de desarrollo. Ambos procesos deben permanecer activos. El comando de compilación genera el frontend, pero por sí mismo no configura un servidor de publicación ni un proxy de producción.

Para detener Spring Boot y Angular se utiliza Ctrl+C. Para detener las bases conservando sus volúmenes:

```
docker compose stop mongodb neo4j
```

Modelo documental de MongoDB

La colección evaluations contiene un documento por contenido de evaluación deduplicado. No se crea un perfil único que sobrescriba los cambios de carrera o procedencia: esos datos permanecen en cada evaluación.

Campo	Significado
_id	SHA-256 del contenido canónico: versión, identidad y los doce campos del CSV
applicant	key, correlativo_aspirante e identityScheme
assignmentDate	Fecha de asignación como BSON Date en UTC, o null
entryYear, birthYear, entryAge	Año de ingreso, año de nacimiento y edad aproximada al ingreso
sex, opportunity, passed	Sexo registrado, número de oportunidad y resultado booleano o desconocido
subject, career	Dimensiones de materia y carrera
origin	Dimensiones department, municipality e institutionType
qualityIssues	Advertencias de calidad y contexto
sources	Hash de archivo, número de registro, año del archivo, variante de encabezado y valores originales
graphSyncState	PENDING, SYNCED o FAILED, según confirmación de sincronización

La identidad es identityScheme:correlativo_aspirante. Los esquemas son full32 para 32 caracteres hexadecimales, short para 20–23 y other para otros formatos. Se compara el identificador completo, sin unir prefijos. Por tanto, los aspirantes distintos son identidades observadas, sin garantía de equivalencia entre formatos históricos.

Las dimensiones conservan key, name y known; carrera y procedencia también conservan sourceValue. La clave municipal incorpora el departamento. Los espacios significativos de carrera, departamento y municipio se preservan para no fusionar textos truncados.

db/mongodb-schema.json define el validador e índices para historiales, carrera/ciclo, materia/resultado, tipo/fecha, geografía y sincronización. La unicidad de evaluaciones corresponde al índice nativo de _id.

La importación inicializa automáticamente las estructuras MongoDB y las restricciones e índices Neo4j desde los recursos de db/. No se requiere ejecutar manualmente esos archivos antes de cargar los CSV.

Colección auxiliar resumen_carrera

M06 consulta una colección materializada: resultados calculados y almacenados para su lectura posterior. Cada documento tiene _id=career.key e incluye evaluaciones, aspirantes únicos, resultados, aspirantes con alguna aprobación, tasa, promedio de edad, muestra y exclusiones. Incluye las categorías desconocidas.

La generación es explícita mediante POST /api/analytics/career-summary/refresh. MongoDB agrupa por carrera y aspirante y luego por carrera; $out escribe una colección temporal propia. Se compara una huella del origen antes y después y, si permanece estable, renameCollection publica el conjunto completo. analytics_metadata conserva los metadatos de publicación.

generatedAt, sourceSnapshot y sourceEvaluationCount identifican el corte. GET no reconstruye el resumen; summary.refreshRequired avisa si el origen cambió. La actualización es global, aunque luego se filtre su lectura por carrera. Si una importación concurrente cambia la huella, devuelve 409 y conserva el resumen anterior. Se debe esperar a que terminen las cargas y volver a generar. Este procedimiento no cambia las evaluaciones.

Modelo de grafos de Neo4j

Etiqueta	Propiedades principales
Applicant	key, correlativo_aspirante, identityScheme
Evaluation	key igual al _id documental, entryYear, assignmentDate, opportunity, passed, entryAge, birthYear, sex, qualityIssues
Subject, InstitutionType	key, name
Career, Department	key, name, sourceValue
Municipality	key, name, sourceValue, departmentKey

Las relaciones, sin propiedades propias, son:

```
(Applicant)-[:TOOK]->(Evaluation)
(Evaluation)-[:OF_SUBJECT]->(Subject)
(Evaluation)-[:TARGETS]->(Career)
(Evaluation)-[:FROM_INSTITUTION_TYPE]->(InstitutionType)
(Evaluation)-[:FROM_MUNICIPALITY]->(Municipality)
(Municipality)-[:IN_DEPARTMENT]->(Department)
```

db/neo4j-schema.cypher declara siete restricciones de unicidad sobre key, una por etiqueta, e índices de evaluación para (entryYear, passed) y (entryYear, opportunity). La importación usa MERGE para reutilizar nodos y relaciones. Las dimensiones desconocidas no generan nodos; sus valores originales siguen disponibles en MongoDB. El modelo representa tipos de institución, no establecimientos individuales.

Proceso ETL y consistencia

El ETL extrae los CSV, transforma los valores y carga ambos motores. DataImportService coordina la lectura y persistencia; EvaluationTransformer contiene las reglas; ImportFileStore conserva archivos, resultados, rechazos y puntos de recuperación.

1. Se reciben entre uno y diez CSV por envío, de hasta 16 MB cada uno y 160 MB por solicitud. La cola admite hasta veinte operaciones activas.

2. Se conserva una copia y su hash. Los archivos se procesan secuencialmente mediante un trabajador; los registros válidos se almacenan en lotes de 250.

3. Commons CSV lee UTF-8 estricto, admite BOM y aplica formato RFC 4180. Se requieren doce encabezados sin duplicados. Se resuelven por nombre los alias históricos numero_de_fecha_de_evaluaci, departamento_institucion_ed y municipio_institucion_.

4. Se validan columnas, identificador y límites de contenido. Filas con anchura incorrecta, correlativo vacío, caracteres nulos o campos superiores a 4.096 caracteres se rechazan. Un archivo ilegible o con encabezados inválidos no puede completarse normalmente.

5. Fechas inválidas, años no utilizables y valores especiales se conservan mediante null, categorías desconocidas y advertencias. APROBADO se convierte en true, NO APROBADO en false; otro resultado es desconocido. SIN REGISTRO, SIN DATOS, NINGUNO y vacíos no se sustituyen por datos supuestos.

6. La edad es entryYear - birthYear. Se excluye del cálculo si faltan años, queda fuera de 0–100 o el nacimiento es posterior a la asignación. La evaluación se conserva. Materia, sexo y tipo se normalizan a mayúsculas; no se reconstruyen nombres truncados.

7. MongoDB realiza upsert por _id y añade cada procedencia una sola vez por hash de archivo/número de registro. Neo4j recibe las evaluaciones persistidas y confirma cada lote en su propia transacción. Solo entonces MongoDB marca SYNCED y la operación aumenta el contador de procesados.

No existe una transacción distribuida. Si Neo4j falla después de guardar MongoDB, el lote documental se conserva y queda FAILED o PENDING; el resultado de importación indica el fallo parcial. Reintentar vuelve a leer la copia conservada y repite operaciones idempotentes, es decir, sin crear duplicados por repetirlas. Un fallo de escritura MongoDB también puede dejar confirmaciones parciales, recuperables por el mismo mecanismo.

Al reiniciar, las operaciones en cola continúan y las que estaban procesándose quedan INTERRUPTED, disponibles para reintento. Respaldar BD2_IMPORT_DIRECTORY es necesario para conservar historial y copias. Los rechazos son distintos de las advertencias de registros válidos. El contador procesado mide filas confirmadas en ambos motores, incluyendo repeticiones; no equivale a documentos nuevos.

Consultas MongoDB

Las agregaciones están en features/analytics/repository/AnalyticsPipelines.java; AnalyticsRepository las ejecuta y AnalyticsService valida el contrato y coordina M06. Las rutas siguientes son GET bajo /api/analytics/.

ID	Ruta	Resultado y unidad
M01	applicants-by-institution-type	Aspirantes distintos por tipo de institución
M02	approvals-by-subject	Evaluaciones aprobadas por materia
M03	approved-applicants-by-career-year	Aspirantes con alguna aprobación por carrera/ciclo
M04	approval-rate-by-subject	Evaluaciones y porcentajes por materia
M05	mean-age-by-career	Edad aproximada promedio por aspirante/carrera
M06	career-summary	Resumen global almacenado por

		carrera
M07	mean-age-approved-by-career-institution	Edad promedio de aspirantes con aprobación por carrera/tipo
M08	approved-applicants-by-municipality-career	Aspirantes con aprobación por municipio/carrera
M09	public-evaluations-by-month-subject	Evaluaciones de tipo PUBLICO por mes de asignación/materia
M10	top-careers-age-16-18	Cinco carreras con más aspirantes de edad calculada 16–18
M11	applicant-performance	Evaluaciones y resultados de una identidad completa
M12	applicants-by-sex-institution	Aspirantes distintos por sexo/tipo
M13	approval-rate-by-age	Porcentajes de evaluaciones por edad calculada
M14	mean-observed-attempts-by-subject	Promedio de oportunidades distintas por aspirante/ciclo/materia
M15	applicant-history	Historial completo con contexto, fuentes y advertencias
M16	first-attempt-failures-by-career	Aspirantes con reprobación en oportunidad literal 1 por carrera
M17	top-municipalities	Cinco municipios con más aspirantes distintos
M18	approval-rate-by-institution-type	Porcentajes de evaluaciones por tipo de institución
M19	career-demand-by-department	Aspirantes distintos y posición de carreras por departamento
M20	approval-evolution-by-entry-year	Evaluaciones y porcentajes por año de ingreso

Los porcentajes usan resultados conocidos como denominador; sin resultados conocidos se devuelve null. Las consultas de edad informan muestra y exclusiones. Para los promedios por aspirante se elige su observación más reciente dentro del grupo y filtro. Las oportunidades distintas no se sustituyen por el ordinal máximo. Sumar aspirantes de diferentes grupos puede contar la misma identidad varias veces.

Se admiten filtros de ciclo, dimensiones, sexo, resultado, edad y fechas de asignación; M06 solo permite carrera, paginación y ordenamiento. M11/M15 exigen correlativo completo. Las páginas admiten hasta cien elementos; los historiales ofrecen nextCursor. El catálogo MongoDB (../consultas/mongodb.md) documenta parámetros y exclusiones.

Los scripts independientes son docs/consultas/mongodb-scripts/M01.js a M20.js. Son ejecutables en mongosh y contienen sus propios filtros. M06 realiza lectura; su bloque de generación permanece comentado. Ejemplo desde la raíz, con .env ya cargado y mongosh disponible:

```
mongosh "mongodb://127.0.0.1:27017/${MONGO_DATABASE}?authSource=admin" \
```

```
--username "$MONGO_USERNAME" --password \
--file docs/consultas/mongodb-scripts/M04.js
```

La contraseña se solicita interactivamente. Si se cambia el nombre de base de la plantilla, revisar también la selección de base de los scripts.

Consultas Neo4j

Las cinco consultas se encuentran en bd2-api/src/main/resources/graph/. GraphRepository las ejecuta con parámetros y transacciones de lectura; GraphService valida filtros y transforma la respuesta. Todas requieren entryYear y se publican como GET bajo /api/graph/.

ID	Ruta y archivo .cypher	Alcance
N01	applicant-history	Historial de una applicantKey completa por ciclo; materia opcional
N02	approval-by-subject-career	Evaluaciones y tasas por materia/carrera conocida; filtros opcionales de ambas
N03	top-careers-by-department	Cinco carreras por aspirantes distintos; departamento obligatorio y municipio opcional
N04	improved-outcomes	Parejas de reprobación y aprobación con oportunidad mayor, misma identidad/materia/ciclo; materia obligatoria
N05	municipality-peers	Caminos entre aspirantes a través de evaluaciones y municipio; identidad obligatoria y municipio opcional

N05 recorre cuatro relaciones y admite hasta 25 caminos por página, con un límite superior de 125 nodos y cien aristas. La respuesta incluye el subgrafo deduplicado de esa página. N01–N04 devuelven filas. Las páginas usan offset, limit y hasMore; no representan el grafo completo. Las consultas tienen un tiempo máximo de diez segundos. El catálogo Neo4j (../consultas/neo4j.md) describe los recorridos.

Endpoints e interfaz

Operación	Endpoint	Pantalla
Comprobar servicios	GET /api/status?target=all	Estado de conexión (/status)
Enviar CSV	POST /api/imports, multipart files	Carga CSV (/data-import)
Consultar cargas	GET /api/imports y GET /api/imports/{id}	Resultados y progreso por archivo
Consultar rechazos	GET /api/imports/{id}/rejections	Diálogo Ver rechazos

Reintentar	POST /api/imports/{id}/retry	Acciones de importación
Consultar estadísticas	GET /api/analytics/{ruta}	Inicio (/) y Consultas MongoDB (/analytics)
Actualizar resumen	POST /api/analytics/career-summary/refresh	Botón de M06
Consultar relaciones	GET /api/graph/{ruta}	Grafos Neo4j (/graph)
Resolver opciones de filtros	GET /api/search/options	Selectores de dimensiones
Buscar evaluaciones	GET /api/search/evaluations	Buscador (/search)

Los errores de parámetros producen 400; recursos inexistentes, 404; motores inaccesibles o consultas que no pueden completarse, 503. La API responde con mensajes controlados. Las listas y búsquedas están paginadas.

Verificación y problemas comunes

Con los procesos iniciados se puede verificar sin escribir datos:

```
curl 'http://localhost:8080/api/status?target=all'
```

```
curl 'http://localhost:8080/api/analytics/approval-rate-by-subject'
```

```
curl
'http://localhost:8080/api/graph/approval-by-subject-career?entryYear=2024&limit=25
'
```

El estado esperado es UP para API y ambos motores. Abrir Inicio, consultar estadísticas y ejecutar N02 permite comprobar la comunicación completa de lectura. Un resultado vacío no demuestra una conexión fallida: puede no haber datos para el filtro.

Los comandos del repositorio para pruebas y empaquetado son, desde bd2-api/, con variables y motores disponibles:

```
./mvnw test
```

```
./mvnw -DskipTests clean package
```

Desde bd2-client/:

```
npm test -- --watch=false
```

```
npm run build
```

Las pruebas completas del backend incluyen importaciones de muestras y escritura de resúmenes; no deben ejecutarse como una comprobación de solo lectura sobre una base que se quiera mantener intacta. Si se ejecuta el backend desde un JAR, detenerlo antes de clean package y reiniciarlo después con el artefacto generado.

datas/analisis/profile_dataset.py solo lee los CSV. verify_analytics.py contrasta estadísticas, pero también reconstruye el resumen mediante POST; verify_graph.py consulta los originales y realiza GET sin importar datos. Los scripts de navegador están en bd2-client/e2e/; requieren un entorno de navegador adicional, que no es un requisito de instalación de la aplicación.

Situación	Revisión o solución
API no inicia por configuración	Exportar .env, comprobar Java 21 y disponibilidad del puerto 8080
MongoDB o Neo4j inaccesible	Revisar docker compose ps, credenciales y puertos; .env no cambia contraseñas de volúmenes existentes
Angular abre, pero no consulta	Verificar que Spring Boot esté activo y usar npm start con el proxy del proyecto
Importación parcial o interrumpida	Recuperar los servicios y pulsar Reintentar conservando el almacén local
CSV inválido	Revisar UTF-8, doce encabezados y motivo mostrado; cargar una copia corregida como archivo nuevo
Historial de importaciones ausente	Comprobar que BD2_IMPORT_DIRECTORY apunta al mismo directorio absoluto y que sus archivos se conservaron
Resumen vacío o desactualizado	Finalizar las cargas y usar Generar / actualizar resumen en M06
Consulta lenta o sin resultados	Acotar filtros y verificar claves completas, ciclo y categorías observadas

La revisión documental comprobó mediante GET la disponibilidad de ambos motores y respuestas analíticas reales. No ejecutó importaciones, reconstrucciones ni la batería de pruebas con escritura.